Universidad de Caldas
Bases de datos relacionales
Plantilla Entrega 1. Proyecto Final

Esta plantilla corresponde a la Entrega 1: Diseño y Configuración del Sistema Híbrido Relacional-Vectorial para RAG. Reemplacen todo texto entre corchetes [ ], conserven los títulos y la numeración, y no eliminen la sección final de evaluación (la diligencia el docente).

- Entreguen en PDF con el nombre E1_Relacionales_[Apellido1]_[Apellido2]_[Apellido3].pdf.
- El diagrama ERD debe ser legible al 100 % de zoom; si es grande, adjunten además la imagen en alta resolución o el archivo fuente.
- Todo script SQL debe estar también en el repositorio del grupo.
- Mínimo de entidades en el ERD: 12 en pareja, 15 en terna.

Campo	**Información del grupo**
Nombre del proyecto	[Music Statistics]
Dominio de datos	[Contenido multimedia musical y preferencias de los usuarios]
Modalidad	[Pareja]
Entidades mínimas exigidas	[12 / 15]
Entidades en el ERD entregado	[32]
Integrante 1 (nombre · código · correo)	[Miguel Angel Ospina Giraldo-52647-miguel.ospina52647@ucaldas.edu.co]
Integrante 2 (nombre · código · correo)	[Yeimy Johana Peña Narváez-59500-yeimy.pena59500@ucaldas.edu.co]
Integrante 3 (nombre · código · correo)	[ ]
Repositorio Git	https://github.com/Miguel4577/estadisticas_musicales

Entorno elegido	[Supabase + Colab / Docker Postgres + pgvector]
Fecha de entrega	[ ]

- **Universo del discurso y análisis de requerimientos**

1.1 Descripción del dominio

[En 1 a 2 párrafos: qué organización o contexto modela el sistema, qué problema resuelve el RAG y quién lo usa. Ej.: "Una red de museos necesita que visitantes e investigadores consulten en lenguaje natural su archivo de fichas técnicas y fotografías".]

1.2 Actores y fuentes de datos

Actor / fuente	**Descripción**	**Volumen estimado**
[Usuario investigador(oyente)]	[Consulta música mediante lenguaje natural, fragmentos de canciones o portadas. Evalúa las respuestas del RAG y recibe recomendaciones personalizadas.]	[10000 usuarios]
[Fuente de Documentos(letras)]	[Origen del catálogo de canciones en múltiples idiomas (código ISO). Formato estructurado en texto enriquecido para fragmentación (*chunking*).]	[50000 documentos]
[Fuente de imágenes(portadas)]	[Archivos visuales en alta resolución de las portadas de los álbumes. Relacionados directamente con el catálogo de discos para búsquedas visuales.]	[5000 imágenes]
[Fuente de audio(archivos sonoros)]	[Archivos en formato binario o rutas de almacenamiento (.mp3, .wav) que contienen las canciones para extracción de características acústicas.]	[50000 archivos]

1.3 Requerimientos funcionales

ID	Requerimiento	Tipo de consulta que lo satisface
RF-01	[Buscar canciones por el idioma original de su letra y filtrar por un rango de fechas de lanzamiento del álbum.]	SQL clásica
RF-02	[Encontrar fragmentos de letras de canciones semánticamente similares a una frase o pregunta en lenguaje natural escrita por el usuario.]	Vectorial texto-texto
RF-03	[Encontrar portadas de álbumes musicalmente afines a partir de una descripción textual o concepto visual provisto por el usuario.]	Vectorial texto→imagen
RF-04	[Buscar canciones combinando criterios tradicionales (artista o género específico) con una búsqueda de similitud sobre los fragmentos de letras.]	Híbrida
RF-05	[Generar una respuesta explicativa y contextualizada (vía LLM) justificando por qué ciertas canciones coinciden con la consulta del usuario.]	RAG
RF-06	[Registrar y consultar las métricas de calidad de las respuestas (fidelidad, relevancia y precisión) generadas por el sistema RAG.]	SQL analitica
RF-07	[Encontrar canciones acústicamente similares a partir de un archivo de audio de entrada mapeado en el espacio vectorial.]	Vectorial audio → audio
RF-08	[Generar una lista de canciones recomendadas para un usuario específico basadas en su historial de	SQL analítica / Recomendación

	reproducciones e interacciones previas.]	

1.4 Requerimientos no funcionales

- [**Rendimiento**: Las búsquedas vectoriales (texto-texto, texto-imagen y audio-audio) utilizando índices HNSW sobre la extensión pgvector deben retornar resultados en un tiempo menor a 1.5 segundos bajo una carga típica de catálogo masivo.]
- [**Integridad**: El sistema debe garantizar la integridad referencial y de dominio mediante restricciones estrictas de llaves primarias (**PK**), llaves foráneas (**FK**), valores únicos (**UNIQUE**) para correos/códigos ISO, campos obligatorios (**NOT NULL**) y cláusulas **CHECK** para asegurar que los scores de similitud, métricas RAG y duraciones estén en los rangos correctos.]
- [**Seguridad y Modularidad**:Las consultas anónimas deben permitirse marcando la relación con el usuario como anulable (`NULL`), pero las interacciones de personalización y recomendaciones requieren una sesión de usuario válida y activa.]
- [**Reproducibilidad**: Toda la infraestructura de la base de datos (PostgreSQL, extensión pgvector, esquemas, tablas e índices vectoriales) se debe desplegar, configurar y poblar automáticamente mediante un único script de orquestación (docker-compose.yml).]

1.5 Supuestos y restricciones del dominio

[-Estrategias de fragmentación: Una letra de canción (`LETRA`) puede ser dividida en múltiples fragmentos (`FRAGMENTO_LETRA`) bajo diferentes metodologías de chunking de forma simultánea. Se restringe que la combinación de la letra, la estrategia y el número de fragmento sea estrictamente única.
-Unicidad de portadas: Un álbum musical puede estar asociado a múltiples imágenes, pero únicamente **una sola imagen** puede ser marcada como la portada principal activa del álbum a la vez (`es_portada_principal = TRUE`).

-Modelos e índices vectoriales: Se asume que los vectores guardados en una misma columna pertenecen al mismo modelo embedding (ej. dimensión 384 para texto y dimensión 512 para imágenes). El sistema prohíbe explícitamente operaciones de comparación o cálculo de distancia entre vectores de dimensiones e industrias incompatibles.

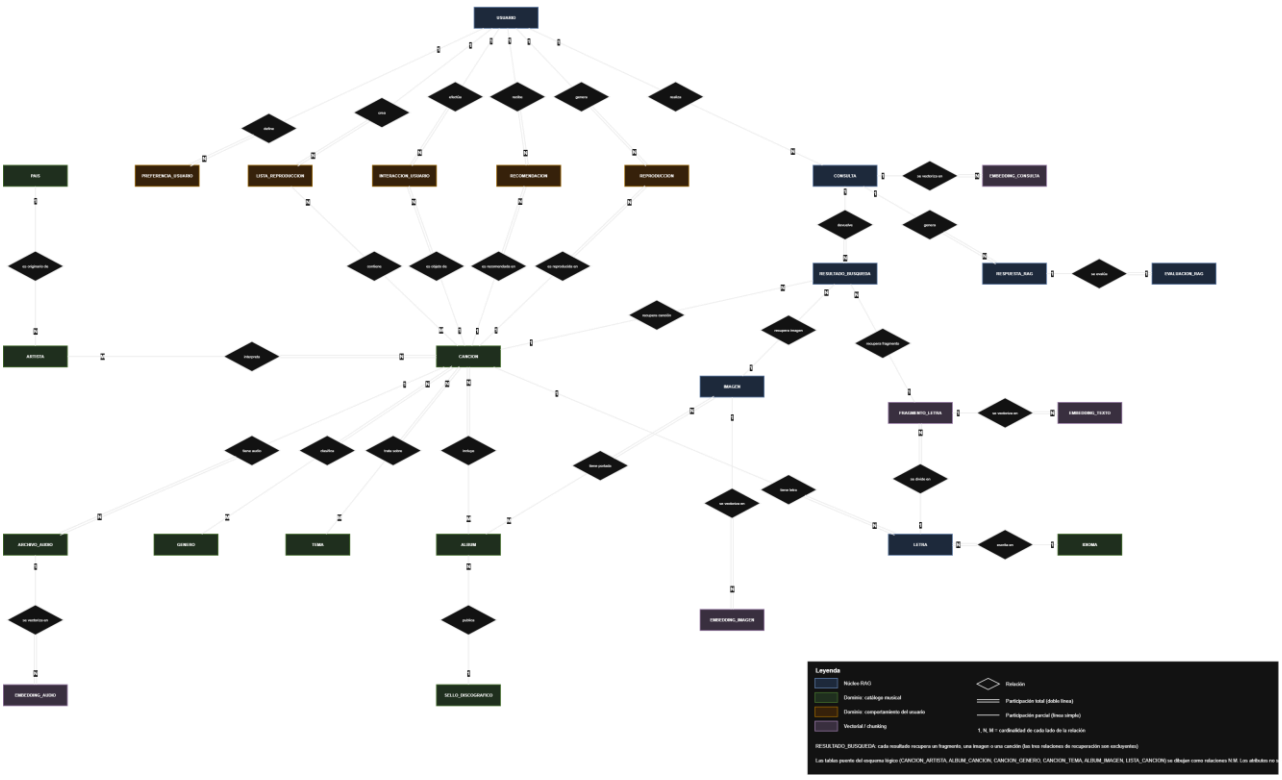
-Exclusividad del resultado: En la tabla de resultados de búsqueda (`RESULTADO_BUSQUEDA`), un registro individual debe representar la recuperación de **exactamente un fragmento de texto o una** imagen, aplicando una restricción de exclusión mutua para evitar registros huérfanos o ambiguos.**]**

- **Diseño conceptual (ERD)**

2.1 Diagrama entidad-relación

Notación Chen

https://app.diagrams.net/#G15amXQPsHzebDszA4mdFackXKHTQWJCns#%7B%22pageId%22%3A%22erd%22%7D



2.2 Catálogo de entidades

El ERD debe incluir las entidades del núcleo RAG (Usuario, Documento, Imagen, Consulta, Resultado, Evaluación) y las propias del dominio hasta completar el mínimo. Marquen el tipo de cada una.

#	Entidad	Tipo	Descripción breve	Atributo identificador
1	USUARIO	Núcleo	Persona registrada que busca, escucha y guarda música	id_usuario
2	LETRA	Núcleo	Texto de una canción en un idioma y versión (el "documento"	id_letra

			del RAG)	
3	IMAGEN	Núcleo	Imagen registrada, usada como portada de álbumes	id_imagen
4	CONSULTA	Núcleo	Pregunta en lenguaje natural, de un usuario o anónima	id_consulta
5	RESULTADO_BUSQUEDA	Núcleo	Elemento recuperado por una consulta, con puntaje y posición	id_resultado
6	RESPUESTA_RAG	Núcleo	Respuesta generada por un LLM con el contexto recuperado	id_respuesta
7	EVALUACION_RAG	Núcleo	Métricas de calidad de una respuesta	id_evaluacion
8	CANCION	Dominio	Obra musical individual	id_cancion
9	ARTISTA	Dominio	Solista, banda u otro intérprete	id_artista
10	ALBUM	Dominio	Lanzamiento que agrupa canciones	id_album
11	SELLO_DISCOGRAFICO	Dominio	Casa discográfica que publica álbumes	id_sello
12	GENERO	Dominio	Género musical	id_genero
13	TEMA	Dominio	Tema o temática de una canción	id_tema
14	IDIOMA	Dominio	Idioma de una letra, con código ISO	id_idioma

15	PAIS	Dominio	País de origen de un artista	id_pais
16	ARCHIVO_AUDIO	Dominio	Archivo de audio de una canción	id_audio
17	REPRODUCCION	Dominio (usuario)	Escucha de una canción por un usuario	id_reproduccion
18	INTERACCION_USUARIO	Dominio (usuario)	Acción de un usuario sobre una canción	id_interaccion
19	RECOMENDACION	Dominio (usuario)	Canción sugerida a un usuario, con puntaje y motivo	id_recomendacion
20	LISTA_REPRODUCCION	Dominio (usuario)	Lista de canciones creada por un usuario	id_lista
21	PREFERENCIA_USUARIO	Dominio (usuario)	Preferencia declarada o inferida, con peso	id_preferencia
22	FRAGMENTO_LETRA	Vectorial	Chunk de una letra según una estrategia de chunking	id_fragmento
23	EMBEDDING_TEXTO	Vectorial	Vector de un fragmento de letra (384)	id_embedding_texto
24	EMBEDDING_IMAGEN	Vectorial	Vector de una imagen (512)	id_embedding_imagen
25	EMBEDDING_AUDIO	Vectorial	Vector de un archivo de audio	id_embedding_audio
26	EMBEDDING_CONSULTA	Vectorial	Vector de una consulta (384)	id_embedding_consulta

2.3 Relaciones y cardinalidades

Relación	Entidad A	Entidad B	Cardinalidad	Participación (A / B)	Regla de negocio
realiza	USUARIO	CONSULTA	1:N	parcial / parcial	Un usuario hace varias consultas; una consulta puede ser anónima
define	USUARIO	PREFERENCIA_USUARIO	1:N	parcial / total	Toda preferencia pertenece a un usuario
crea	USUARIO	LISTA_REPRODUCCION	1:N	parcial / total	Toda lista tiene un creador
efectúa	USUARIO	INTERACCION_USUARIO	1:N	parcial / total	Toda interacción la hace un usuario
recibe	USUARIO	RECOMENDACION	1:N	parcial / total	Toda recomendación es para un usuario
genera	USUARIO	REPRODUCCION	1:N	parcial / total	Toda reproducción la hace un usuario
contiene	LISTA_REPRODUCCION	CANCION	N:M (vía LISTA_CANCION)	parcial / parcial	Una lista agrupa varias canciones y una canción está en varias listas
es objeto de	CANCION	INTERACCION_USUARIO	1:N	parcial / total	Toda interacción es sobre una canción
es recomendada en	CANCION	RECOMENDACION	1:N	parcial / total	Toda recomendación apunta a

					una canción
es reproducida en	CANCION	REPRODUCCION	1:N	parcial / total	Toda reproducción es de una canción
se vectoriza en	CONSULTA	EMBEDDING_CONSULTA	1:N	parcial / total	Una consulta puede tener un vector por modelo
devuelve	CONSULTA	RESULTADO_BUSQUEDA	1:N	parcial / total	Todo resultado pertenece a una consulta
genera	CONSULTA	RESPUESTA_RAG	1:N	parcial / total	Una consulta puede generar varias respuestas (pruebas con distintos modelos)
se evalúa	RESPUESTA_RAG	EVALUACION_RAG	1:1	parcial / total	Cada respuesta tiene a lo sumo una evaluación
recupera canción	RESULTADO_BUSQUEDA	CANCION	N:1	parcial / parcial	Un resultado recupera exactamente una entidad: canción, fragmento o imagen
recupera fragmento	RESULTADO_BUSQUEDA	FRAGMENTO_LETRA	N:1	parcial / parcial	Ídem
recupera imagen	RESULTADO_BUSQUEDA	IMAGEN	N:1	parcial / parcial	Ídem
es originario de	ARTISTA	PAIS	N:1	parcial / parcial	Un artista tiene a lo sumo un país de origen

interpreta	ARTISTA	CANCION	N:M (vía CANCION_ARTISTA)	parcial / total	Toda canción tiene al menos un artista, con un rol
incluye	ALBUM	CANCION	N:M (vía ALBUM_CANCION)	parcial / total	Una canción aparece en uno o más álbumes, con número de pista
publica	SELLO_DISCOGRAFICO	ALBUM	1:N	parcial / parcial	Un álbum tiene a lo sumo un sello
clasifica	GENERO	CANCION	N:M (vía CANCION_GENERO)	parcial / total	Toda canción tiene al menos un género
trata sobre	TEMA	CANCION	N:M (vía CANCION_TEMA)	parcial / total	Toda canción tiene al menos un tema, con método de asignación
tiene letra	CANCION	LETRA	1:N	parcial / total	Toda letra pertenece a una canción
escrita en	IDIOMA	LETRA	1:N	parcial / total	Toda letra tiene un idioma
se divide en	LETRA	FRAGMENTO_LETRA	1:N	parcial / total	Toda letra se parte en fragmentos para el RAG
se vectoriza en	FRAGMENTO_LETRA	EMBEDDING_TEXTO	1:N	parcial / total	Un fragmento puede tener un vector por modelo
tiene	ALBUM	IMAGEN	N:M (vía	parcial /	Toda imagen

portada			ALBUM_IMAGEN)	total	está asociada a un álbum; una es la portada principal
se vectoriza en	IMAGEN	EMBEDDING_IMAGEN	1:N	parcial / total	Una imagen puede tener un vector por modelo
tiene audio	CANCION	ARCHIVO_AUDIO	1:N	parcial / total	Todo archivo de audio es de una canción
se vectoriza en	ARCHIVO_AUDIO	EMBEDDING_AUDIO	1:N	parcial / total	Un audio puede tener un vector por modelo

- **Diseño lógico en 3FN**

3.1 Esquema relacional

Una fila por tabla. Subrayen o marquen la PK y anoten cada FK con la tabla que referencia.

Tabla	PK	FK (→ tabla referenciada)	Atributos	Restricciones relevantes
pais	id_pais	—	nombre	nombre UNIQUE, NOT NULL
sello_discografico	id_sello	—	nombre, sitio_web	nombre UNIQUE, NOT NULL
genero	id_genero	—	nombre, descripcion	nombre UNIQUE, NOT NULL
tema	id_tema	—	nombre, descripcion	nombre UNIQUE, NOT NULL
idioma	id_idioma	—	nombre, codigo_iso	nombre y codigo_iso UNIQUE, NOT NULL
artista	id_artista	id_pais → pais	nombre_artistico, tipo_artista,	tipo_artista CHECK (solista, banda, otro);

			fecha_inicio, fecha_fin	fecha_fin ≥ fecha_inicio
album	id_album	id_sello → sello_discografico	titulo, fecha_lanzamiento, tipo_lanzamiento	titulo NOT NULL
cancion	id_cancion	—	titulo, duracion_segundos, es_explicita, fecha_lanzamiento	duracion_segundos CHECK > 0; es_explicita DEFAULT FALSE
imagen	id_imagen	—	ruta_archivo, descripcion, texto_alternativo, fuente, fecha_registro	ruta_archivo NOT NULL
archivo_audio	id_audio	id_cancion → cancion	ubicacion_archivo, formato, duracion_segundos, frecuencia_muestreo	duracion_segundos CHECK > 0
cancion_artista	(id_cancion, id_artista, rol)	id_cancion → cancion; id_artista → artista	rol	rol NOT NULL
album_cancion	(id_album, id_cancion)	id_album → album; id_cancion → cancion	numero_pista	UNIQUE(id_album, numero_pista); numero_pista CHECK > 0
cancion_genero	(id_cancion, id_genero)	id_cancion → cancion; id_genero → genero	—	—
cancion_tema	(id_cancion, id_tema)	id_cancion → cancion; id_tema → tema	metodo_asignacion	metodo_asignacion CHECK (manual, metadatos, automatico)
album_imagen	(id_album, id_imagen)	id_album → album; id_imagen → imagen	es_portada_principal	una sola portada principal por álbum (índice único parcial)
letra	id_letra	id_cancion → cancion; id_idioma → idioma	texto, fuente, version	UNIQUE(id_cancion, id_idioma, version); texto NOT NULL

fragmento_letra	id_fragmento	id_letra → letra	numero_fragmento, texto_fragmento, estrategia_chunking, token_inicio, token_fin	UNIQUE(id_letra, estrategia_chunking, numero_fragmento); numero_fragmento CHECK > 0
embedding_texto	id_embedding_texto	id_fragmento → fragmento_letra	modelo, version_modelo, vector_embedding VECTOR(384), fecha_generacion	único por (fragmento, modelo, versión)
embedding_imagen	id_embedding_imagen	id_imagen → imagen	modelo, vector_embedding VECTOR(512), fecha_generacion	UNIQUE(id_imagen, modelo)
embedding_audio	id_embedding_audio	id_audio → archivo_audio	modelo, version_modelo, vector_embedding VECTOR(N), fecha_generacion	único por (audio, modelo, versión)
usuario	id_usuario	—	nombre, email, fecha_registro, zona_horaria, idioma_preferido	email UNIQUE, NOT NULL
preferencia_usuario	id_preferencia	id_usuario → usuario	tipo_preferencia, valor_preferencia, peso	peso CHECK entre 0 y 1
lista_reproduccion	id_lista	id_usuario → usuario	nombre, descripcion, fecha_creacion, es_publica	nombre NOT NULL
lista_cancion	(id_lista, id_cancion)	id_lista → lista_reproduccion; id_cancion → cancion	posicion, fecha_agregada	UNIQUE(id_lista, posicion); posicion CHECK > 0
reproduccion	id_reproduccion	id_usuario → usuario; id_cancion → cancion	fecha_hora_inicio, duracion_escuchada_segundos, completada, origen_reproduccion	duracion_escuchada_segundos CHECK ≥ 0
interaccion_usuario	id_interaccion	id_usuario → usuario; id_cancion → cancion	tipo_interaccion, fecha_hora	tipo_interaccion NOT NULL

recomendacion	id_recomendacion	id_usuario → usuario; id_cancion → cancion	fecha_hora, puntaje, motivo, estado	estado NOT NULL
consulta	id_consulta	id_usuario → usuario (NULL permitido)	texto_pregunta, fecha_consulta, tipo_consulta	tipo_consulta CHECK (texto, imagen, hibrida)
embedding_consulta	id_embedding_consulta	id_consulta → consulta	modelo, vector_embedding VECTOR(384), fecha_generacion	UNIQUE(id_consulta, modelo)
resultado_busqueda	id_resultado	id_consulta → consulta; id_fragmento → fragmento_letra; id_imagen → imagen; id_cancion → cancion	tipo_recuperacion, score_similitud, posicion	exactamente uno de id_fragmento, id_imagen, id_cancion no nulo; UNIQUE(id_consulta, posicion)
respuesta_rag	id_respuesta	id_consulta → consulta	texto_respuesta, modelo_llm, fecha_generacion, tiempo_respuesta_ms	tiempo_respuesta_ms ≥ 0
evaluacion_rag	id_evaluacion	id_respuesta → respuesta_rag	faithfulness, answer_relevancy, context_recall, context_precision, modelo_evaluacion, fecha_evaluacion	id_respuesta UNIQUE (1:1); métricas entre 0 y 1

3.2 Justificación de la normalización

- 1FN: No hay atributos multivaluados ni grupos repetitivos. Los multivaluados del dominio (artistas, géneros, temas y álbumes de una canción; portadas de un álbum; canciones de una lista) se resolvieron con tablas puente. Los fragmentos de una letra y los vectores de un objeto viven en filas propias (fragmento_letra, embedding_*), no en columnas repetidas.
- 2FN: en las tablas de PK compuesta (cancion_artista, album_cancion, cancion_genero, cancion_tema, album_imagen, lista_cancion) cada atributo no clave depende de la clave completa: rol depende del trío (canción, artista, rol), numero_pista de (álbum, canción), metodo_asignacion de (canción, tema), es_portada_principal de (álbum,

imagen), y posicion / fecha_agregada de (lista, canción).

- 3FN: no existen dependencias transitivas. Ejemplos de las que se eliminaron: el país del artista, el sello del álbum y el idioma de la letra se guardan como llaves a catálogos (pais, sello_discografico, idioma) y no como texto repetido; la evaluación se separó de la respuesta para que las métricas no dependan transitivamente de la consulta; y el nombre del usuario no se repite en consulta, reproducción ni lista.
-

3.3 Decisiones de diseño

- resultado_busqueda usa tres FK opcionales con un CHECK de exactamente una no nula, en lugar de una tabla por tipo de resultado: así toda consulta tiene un único ranking ordenado (posición) que mezcla fragmentos, imágenes y canciones.
- Los embeddings viven en tablas separadas de los datos originales: cada modalidad tiene su propia dimensión (384 / 512 / N) y se puede guardar más de un modelo por objeto sin alterar las tablas base.
- fragmento_letra guarda la estrategia de chunking como atributo para conservar en la misma tabla los fragmentos de las tres estrategias del experimento.
- idioma, país, género, tema y sello son tablas catálogo, no atributos, para evitar duplicados y permitir restricciones UNIQUE.
- consulta.id_usuario admite NULL para consultas anónimas; al borrar un usuario sus consultas se conservan sin dueño.
- respuesta_rag tiene PK propia (no id_consulta) porque una consulta puede generar varias respuestas al probar distintos LLM o configuraciones.

4. Identificación de campos vectorizables

Campo origen	Tabla origen	Modalidad	Modelo de embedding	Dimensión	Tabla destino	Justificación
texto_fragmento (por chunk)	fragmento_letra	Texto	all-MiniLM-L6-v2	384	embedding_texto	Permite buscar letras por

						significado (RF-02, RF-04) en lugar de por palabras exactas
texto_pregunta	consulta	Texto	all-MiniLM-L6-v2	384	embedding_consulta	Debe estar en el mismo espacio que los fragmentos para compararlos
Archivo de imagen (píxeles)	imagen	Imagen	CLIP ViT-B/32	512	embedding_imagen	Representa el contenido visual de la portada (RF-03)
texto_pregunta (para texto→imagen)	consulta	Texto	CLIP ViT-B/32 (codificador de texto)	512	[vector 512 calculado en la consulta, sin tabla propia o en embedding_consulta con otra dimensión]	Comparar una descripción con imágenes requiere el espacio de CLIP
Archivo de audio	archivo_audio	Audio	[modelo de audio, p. ej. CLAP]	N (512 de ejemplo)	embedding_audio	Búsqueda por similitud sonora (RF-09)
descripcion / texto_alternativo	imagen	Texto	[CLIP texto / MiniLM]	[512 / 384]	[Opcional]	[Solo si se quiere buscar imágenes por su descripción]

- El embedding de imagen se calcula sobre el contenido visual del archivo, cargado con PIL y procesado por CLIP; no se vectoriza la cadena de texto de ruta_archivo.
- Búsqueda texto→imagen: la consulta se codifica con el codificador de texto de CLIP (512 dimensiones), no con MiniLM (384), de modo que el vector de la pregunta y los de embedding_imagen quedan en el mismo espacio. [Definan si ese vector se guarda: embedding_consulta tiene VECTOR(384) fijo, así que haría falta una segunda tabla o columna de 512.]
- Métrica de distancia elegida: coseno (vector_cosine_ops, operador <=>). Los modelos MiniLM y CLIP están pensados para similitud coseno, y esa métrica ignora la magnitud del vector y compara solo su dirección.
-

5. Esquema SQL (tablas relacionales y vectoriales)

Peguen aquí el DDL completo o, si es extenso, las tablas vectoriales y el enlace al archivo `schema.sql` del repositorio. El script debe ejecutarse sin errores sobre una base vacía.

Archivo en el repositorio: [ruta/schema.sql]

Referencia mínima esperada para las tablas vectoriales (adapten nombres y tipos a su diseño):

```
CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE fragmento_letra (
    id_fragmento        BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_letra            INTEGER NOT NULL REFERENCES letra(id_letra) ON DELETE
CASCADE,
    numero_fragmento    INTEGER NOT NULL CHECK (numero_fragmento > 0),
    texto_fragmento     TEXT NOT NULL,
    estrategia_chunking VARCHAR(30) NOT NULL,
    token_inicio        INTEGER,
    token_fin           INTEGER,
    UNIQUE (id_letra, estrategia_chunking, numero_fragmento)
);

CREATE TABLE embedding_texto (
    id_embedding_texto BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_fragmento       BIGINT NOT NULL REFERENCES fragmento_letra(id_fragmento) ON
DELETE CASCADE,
```

```
    modelo              VARCHAR(100) NOT NULL,
    version_modelo      VARCHAR(50),
    vector_embedding    VECTOR(384) NOT NULL,
    fecha_generacion    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE embedding_imagen (
    id_embedding_imagen BIGINT GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    id_imagen           INTEGER NOT NULL REFERENCES imagen(id_imagen) ON DELETE
CASCADE,
    modelo              VARCHAR(100) NOT NULL,
    vector_embedding    VECTOR(512) NOT NULL,
    fecha_generacion    TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
    UNIQUE (id_imagen, modelo)
);

CREATE INDEX idx_hnsw_embedding_texto
    ON embedding_texto USING hnsw (vector_embedding vector_cosine_ops);
CREATE INDEX idx_hnsw_embedding_imagen
    ON embedding_imagen USING hnsw (vector_embedding vector_cosine_ops);
```

- **Entorno ejecutable**

Elemento	Valor
Opción	Neon + pgvector
Versión de PostgreSQL	13 o superior
Versión de pgvector	0.5 o superior
Lenguaje y librerías de conexión	[Python · psycopg / SQLAlchemy]
Archivo de arranque	Esta en el git, archivo SQL

7. Plan de experimento de chunking

En este curso se implementan y comparan las tres estrategias. Si modifican los parámetros sugeridos, justifiquen el cambio según su dominio.

7.1 Estrategias y parámetros

Estrategia	Parámetros sugeridos	Parámetros del grupo	Herramienta / librería	Por qué podría funcionar en su dominio
Fixed-size	chunk_size = 256 tokens, overlap = 32	[chunk_size = 64 tokens, overlap = 16]	[langchain TokenTextSplitter]	[Funciona como una línea base simple. Al usar un tamaño de bloque pequeño,

				evita mezclar estrofas muy lejanas de la canción, manteniendo partes del coro compactas.]
Sentence-aware	máx. 5 oraciones, overlap = 1 oración	[máx. 4 líneas/versos, overlap = 1 línea]	[nltk / spaCy + lógica propia]	[Es útil porque la música se escribe en versos y estrofas separadas por saltos de línea en lugar de puntos finales. Cortar cada 4 líneas preserva la estructura musical tradicional (cuartetas).]
Estrategia	Parámetros sugeridos	Parámetros del grupo	Herramienta / librería	Por qué podría funcionar en su dominio
Semantic	umbral coseno entre 0.75 y 0.85	[máx. 4 líneas/versos, overlap = 1 línea]	[semantic-chunkers / propio con MiniLM]	[Es la más prometedora. Identifica cuándo cambia la temática o la emoción de la letra (ej. del verso al coro) y agrupa las líneas según el significado real de lo que el artista está expresando.]

Hipótesis: [A diferencia de los textos académicos, las letras de las canciones no siguen reglas gramaticales rígidas ni usan signos de puntuación tradicionales (puntos o comas) de manera uniforme, lo que invalida la precisión de la estrategia Sentence-aware. Cortar por un tamaño fijo

(Fixed-size) corre el grave riesgo de romper frases o coros a la mitad, dejando palabras clave sin su contexto emocional.

El Semantic Chunking sobresale porque analiza la similitud del significado entre líneas consecutivas; esto garantiza que las estrofas que describen una historia, un sentimiento o una temática específica permanezcan unidas en un solo bloque (chunk), permitiendo que el buscador vectorial conecte de forma exacta las frases en lenguaje natural de los usuarios con el mensaje real de las canciones.]